技术报告 #第一关→找到入口----------

1. 题目详细描述

· 任务背景：在离线 Linux 工作区 /workspace/ 中处理材料。 · 核心要求：需要利用输入文件和密钥，调用程序 /workspace/exam 生成结果文件，最终提交至 /workspace/flag。 · 通关条件：/workspace/flag 必须是大小不超过 1 MiB 的普通文件，且其内容必须与题目预期结果逐字节严格一致。

2. 环境及工具清单

· 基础环境：WSL2 (Ubuntu) · 核心工具：/workspace/exam（题目给定的处理程序） · Linux 命令：ls、cat、cd、ln、cp、mv、echo、file

3. 解题思路

. 进入 /workspace 使用 ls -la 查看目录，发现 input.bin，并在子目录 cabinet 中找到了密钥文件 key.txt。，执行 ./exam --help。程序给出了关键提示：它会硬编码读取 /workspace/input，并将结果硬编码写入 /workspace/output。同时要求用 -e 参数传入非空 ASCII 密钥，准备输入文件，因为程序写死了只认 input 文件名，必须把 input.bin 转化为 input，使用密钥 MOON 执行程序：./exam -e "MOON"。运行成功后，将生成的中间文件 output 重命名为最终要求的 flag：mv /workspace/output /workspace/flag。

4. 我的疑问

· 疑问 1：ln -s 是什么意思？建个快捷方式不是很聪明吗，为什么程序会报错？ · 疑问 2：原本的 input.bin 是不是本来就是一个快捷方式？为什么分不清？ · 疑问 3：我之前没看到 output 文件，是程序没生成吗？

5. 遇到的问题及解决方法

【踩坑经历一】试图用软链接“糊弄”程序

· 错误操作：我最初的想法是把 input.bin 与程序需要的 input 之间建立联系，于是执行了 ln -s /workspace/input.bin /workspace/input。 · 报错现象：程序直接报错退出：exam: /workspace/input 必须是普通文件，不能是符号链接。 · 原因分析：Linux 文件系统中有“普通文件”（占磁盘数据块，权限位开头为 -）和“软链接”（占极小 inode，只为指向目标，权限位开头为 l）。exam 程序出于安全考虑，明确拒绝了链接文件。 · 解决方案：果断删除软链接（rm -f /workspace/input），使用 cp /workspace/input.bin /workspace/input 复制出一个真实的物理普通文件。

【踩坑经历二】寻找 input.bin 的身份

· 困惑：看到程序报“不能是符号链接”，误以为原本的 input.bin 也是链接文件。 · 解决方案：使用 ls -la 查看，确认 input.bin 的权限位开头是 -rw-r--r--（-代表普通文件），意识到“有问题的不是原文件，而是我自作聪明建的快捷方式”。

【踩坑经历三】输出文件找不到

· 现象：运行完程序后，不知道文件去哪了。 · 原因分析：程序是“死脑筋”，它不按我们的心思走，而是固定在 /workspace/output 生成。 · 解决方案：理解了“程序固定输出路径”的概念后，直接在该路径去查看，并用 mv 命令将其重命名为题目要求的 flag，最后用 file 命令确认它确实是普通文件。

#第二关→两段信号----------

1. 题目详细描述

· 任务背景：/workspace/ 目录中夹杂了无关文件，你需要找出两段有效信号并生成一份完整结果。 · 核心要求：通过文件名中的 signal-A、signal-B 标识找到两份输入。从包含 ACCESS_KEY= 的密钥清单中提取唯一密钥。使用 /workspace/exam 分别处理 A 和 B，最终结果必须保持 A 段在前、B 段在后，不能缺段、重复或增加额外字节。 · 通关条件：提交大小不超过 1 MiB 的普通文件 /workspace/flag，其字节内容必须严格等于 A 段处理结果与 B 段处理结果的有序拼接（二进制级比对）。

2. 环境及工具清单

· 基础环境：WSL2 (Ubuntu) · 核心工具：/workspace/exam（题目给定的处理程序） · Linux 命令：find、grep、管道 |、xargs、cut、cat、重定向 >、ls

3. 解题思路（核心流程与细节） . 由于目录可能嵌套多层，使用 find /workspace -type f | grep signal 递归查找文件名中包含 signal 的文件。发现了 A 和 B 的源文件路径。，使用 grep -r "ACCESS_KEY=" /workspace 或 find /workspace -type f | xargs grep -l "ACCESS_KEY=" 在所有文件中搜索密钥行所在的具体文件，由于提示警告“整行文本未必都是密钥”（行内包含干扰内容），使用 grep "ACCESS_KEY=" 文件名 | cut -d'=' -f2 以等号为分隔符截取第二列，提取出干净的密钥，利用程序硬编码读取 /workspace/input 的特性：先将 A 文件复制为 input，执行 exam -e "密钥"。执行完成后立刻将生成的 output 重命名为 result_A，防止被下一次执行覆盖。然后再将 B 文件复制为 input，执行后生成 result_B。使用 cat /workspace/result_A /workspace/result_B > /workspace/flag，严格按照 A 在前、B 在后的顺序进行二进制拼接，最终生成满足通关条件的文件。

4. 我的疑问

· 疑问 1：find、grep、管道 |、重定向 > 分别怎么用？ · 疑问 2：xargs 是什么意思？为什么有时候 find ... | grep 可以，有时候非要写成 find ... | xargs grep？ · 疑问 3：cut 命令是怎么用的？在提取密钥的时候为什么用 cut 就能把等号前面的干扰字符切掉？ · 疑问 4：文件拼接怎么保证“有序”和“二进制严格一致”？

5. 遇到的问题及解决方法

【踩坑经历一】find 与 grep 的混淆

· 困惑：起初不明白为什么 find /workspace -type f | grep signal 不需要加 xargs，而 find ... | xargs grep 却需要。 · 原因分析：理解了两者的区别。find 输出的是文件路径（纯文本），管道 | 传给 grep 时，grep 是在这些文本文字里找 signal 这个词。而 find | xargs grep 是要把找出的文件路径作为参数传给 grep，让 grep 去打开文件并在文件内容里搜索。xargs 充当了“打包整理员”的角色，把标准输入转换为命令行参数。 · 解决方案：区分“文件名过滤”和“文件内容搜索”，掌握了 xargs 的适用场景。

【踩坑经历二】密钥提取时被干扰

· 现象：使用 grep 找出来的行是一整行，里面包含 ACCESS_KEY= 前缀和多余注释文字，无法直接用。 · 解决方案：使用 cut -d'=' -f2，指定等号为分隔符取第二段文字，成功把前缀切掉，提取出真正有用的密钥字符串。

【踩坑经历三】输出文件被覆盖

· 现象：处理完 A 段后没保存，紧接着处理 B 段，导致 A 段的 output 被 B 段覆盖。 · 原因分析：exam 程序硬编码生成 /workspace/output，每次执行都会覆盖旧文件。 · 解决方案：引入变量或手动重命名，处理完 A 段后立刻执行 mv /workspace/output /workspace/result_A，然后再处理 B 段并重命名为 result_B，最后使用 cat 完成拼接。

#第三关→读日志并修复输入----------

1. 题目详细描述

· 任务背景：一份材料未被正确处理。程序 /workspace/exam 对输入文件的字节长度有极其严格的要求，需要依据程序的诊断信息进行修复。 · 核心要求：原始输入位于 /workspace/，文本为 ASCII 且带有一个结尾换行。允许的修改仅为在最后一行文字与结尾换行符之间插入 ASCII 空格，其他所有字节必须原样保留。修复后的输入必须满足程序的要求，且新增空格数必须最少。 · 通关条件：提交 /workspace/flag，内容必须与修复输入后生成的结果逐字节相同。不需要提交修改后的输入文件本身。

2. 环境及工具清单

· 基础环境：WSL2 (Ubuntu) · 核心工具：/workspace/exam（题目给定的处理程序） · Linux 命令：ls、cat、head、printf、cp、echo、od、rm

3. 解题思路（核心流程与细节）

·查看诊断日志，计算差额。执行 cat /var/log/ncuscc/exam.log，看到报错：输入字节数必须是密钥字节数的整数倍（输入 19 字节，密钥 4 字节）。通过计算得知，19 字节向上取最近的 4 的倍数是 20，因此需要补充 1 个 ASCII 空格。 题目明确指出“结尾换行也占用字节”。这意味着文件末尾的结构是 [最后一行文字] + [换行符 \n]。我们要插入空格，必须插在文字和换行符之间，而不是文件最末尾。 禁止使用 vim 或 nano 等文本编辑器，因为编辑器会自动添加或改变换行符。使用纯命令行进行“无创伤”修改：

1. head -c -1 /workspace/input.bin > /workspace/input_fixed：切除原文件末尾的 1 个字节（换行符）。
2. printf " \n" >> /workspace/input_fixed：追加 1 个空格和 1 个换行符。
3. cp /workspace/input_fixed /workspace/input：覆盖程序需要的输入文件。 使用密钥 NOVA 执行 ./exam -e "NOVA"，成功生成 /workspace/output，最后将其重命名为 /workspace/flag。

4. 我的疑问

· 疑问 1：报错 exam: 输入字节数必须是密钥字节数的整数倍（输入 19 字节，密钥 4 字节） 是什么意思？为什么长度不匹配就不能运行？ · 疑问 2：head -c -1 /workspace/原始输入文件名 > /workspace/input_fixed 这个命令是怎么用的？为什么是 -c -1？ · 疑问 3：为什么要用 printf " \n" >> 这种极其别扭的方式，不能直接打开文件用编辑器敲空格吗？

5. 遇到的问题及解决方法

【踩坑经历一】字节数不匹配的困惑

· 现象：直接运行程序报错 19 和 4 无法整除。 · 原因分析：程序要求输入字节数必须是密钥字节数的整数倍。19 除以 4 等于 4 余 3，不满足条件。17 是最近的 4 的倍数（19+1=20）。 · 解决方案：通过日志算出需要补充 1 个字节。掌握了“换行符占 1 个字节”这个底层常识。

【踩坑经历二】空格插错了位置

· 现象：如果直接用 echo " " >> 文件，空格会加在换行符的后面，导致文件结构变成 [文字] + [\n] + [空格]，字节比对依然失败。 · 解决方案：必须先“切开”，再“缝合”。使用 head -c -1 精准切除原有末尾的换行符，再用 printf " \n" 补上“空格+换行符”，完美生成 [文字] + [空格] + [\n] 的正确结构。

【踩坑经历三】编辑器带来的灾难

· 现象：如果试图用 vim 或 nano 打开文件加空格，保存后无论如何都无法通过评测。 · 原因分析：文本编辑器非常“聪明”，它会自动识别文本格式并在末尾添加额外的换行符，或者改变隐藏的编码，导致字节数永远差那么 1-2 个字节。 · 解决方案：在严苛的二进制比对场景下，坚决使用 head 和 printf 这种底层工具，实现对文件字节的“外科手术级”掌控。

#第四关→让命令找到路----------

1. 题目详细描述

· 任务背景：需要编写 /workspace/route.sh，将一个工具的执行结果转换为约定的业务结果。 · 核心要求：脚本接收 MODE（只能是 ok、retry、fatal）和 ITEM（1-32个字母或数字）两个参数。配置文件是 /workspace/job.env，工具目录由 TASK_TOOL_DIR 指定，结果文件路径由 ROUTE_RESULT 指定。工具名为 task-probe，须按命令名调用，能读取配置中的 ACCESS_TOKEN，且不能把工作目录或结果路径写死。 · 通关条件：根据工具 task-probe 的退出码生成不同结果：退出 0 时结果为 receipt:<值>（脚本退出0）；退出 11 时结果为 retry:（脚本退出75）；退出 23 时结果为 rejected:（脚本退出65）。每个结果恰有一行并含结尾换行，需覆盖旧内容。缺参或非法参数时输出 usage、退出 64，且不生成结果文件。

2. 环境及工具清单

· 基础环境：WSL2 (Ubuntu) · 核心工具：/workspace/exam（题目给定的处理程序）或自定义脚本 · Linux 命令：source、export、PATH 环境变量、printf、rm、case 语句、nano 编辑器

3. 解题思路（核心流程与细节）

· 第一步：参数校验与旧文件清理。 检查 $# 是否为 2，检查 1是否匹配$(ok|retry|fatal)$，检查 2是否匹配$[a-zA-Z0-9]1,32$。如果不满足，输出 usage，并使用 rm -f "$ROUTE_RESULT" 清理旧结果文件，最后 exit 64（满足“不生成结果文件”的硬性要求）。 · 第二步：加载配置并导出环境变量。 使用 source /workspace/job.env 加载配置，拿到 ACCESS_TOKEN。关键点： 必须使用 export ACCESS_TOKEN 将其导出为环境变量，这样子进程 task-probe 才能读到（对应提示1：当前 shell 可见的变量不一定对子进程可见）。 · 第三步：设置 PATH 环境变量。 因为工具要求按命令名调用，不能写死绝对路径。执行 export PATH="$TASK_TOOL_DIR:$PATH$”，将工具目录加进$PATH$，保证脚本能直接执行$task-probe$。· 第四步：执行工具并秒存退出码。执行$OUTPUT$ =(task-probe "$MODE" "$ITEM$”)，紧接着必须使用$RC$ =? 立刻保存退出码（对应提示3：非零退出码可能是正常业务状态，后续命令会覆盖 $?）。绝不能使用 set -e，否则遇到 11 或 23 的退出码脚本会直接崩掉。 · 第五步：状态机分支处理。 使用 case $RC in 判断： · 0)：将捕获的输出写入 $ROUTE_RESULT，脚本 exit 0。 · 11)：写入 retry:$ITEM，脚本 exit 75。 · 23)：写入 rejected:$ITEM，脚本 exit 65。 · *)：直接 exit 1。

4. 我的疑问

· 疑问 1：配置文件、工具目录、结果文件路径到底是什么意思？ · 疑问 2：export 到底有什么用？为什么脚本里可见的变量子进程看不见？ · 疑问 3：ROUTE_RESULT 并不在 job.env 里，为什么脚本能使用它？ · 疑问 4：PATH 是什么东西？脚本为什么会找不到 task-probe？ · 疑问 5：if [$# -ne 1]; then echo "usage"; exit 64; fi 这种语法是什么意思？

5. 遇到的问题及解决方法

【踩坑经历一】找不到工具 task-probe

· 现象：执行脚本直接报错 command not found。 · 原因分析：系统默认的 PATH 环境变量里没有 /opt/recruitment/bin 这个目录，系统找不到这个程序。 · 解决方案：使用 export

PATH="$TASK_TOOL_DIR:$PATH"，把工具目录加到系统路径最前面，这样系统一搜就能搜到。

【踩坑经历二】评测失败，报“结果文件内容不符”

· 现象：本地测试明明通过了，但提交到评测系统却报结果内容不对。 · 原因分析：第一，我们使用了 echo "$OUTPUT"，由于 echo 在某些情况下会产生多余的空行或异常换行，导致与标准答案差 1 个字节。第二，我们之前写的是 export ROUTE_RESULT="/workspace/output"，而题目评分要求最终产出必须是 /workspace/flag。 · 解决方案：使用 printf "%s\n" "$OUTPUT" > "$ROUTE_RESULT" 严格保证换行符的字节数恰好为 1 个。同时，将结果路径的兜底逻辑改为 export ROUTE_RESULT="/workspace/flag"，确保 100% 生成正确的文件。

【踩坑经历三】对工具输出的恐惧

· 现象：不确定工具成功时打印的内容是什么，担心提取逻辑写错。 · 解决方案：在终端手动执行 ./exam -e "MOON" 验证输出，确认输出包含 receipt: 前缀。代码中直接用变量 $OUTPUT 接收，原封不动写入文件，避免破坏工具自带的输出格式。 #第五关→照看后台任务----------

1. 题目详细描述

· 任务背景：需要编写 /workspace/manage.sh，为 /opt/recruitment/bin/slow-worker 提供可重复使用的进程管理接口。 · 核心要求：脚本接收 start、status、report、stop 四个参数。工作目录由 WORK_DIR 指定，PID 记录位于 $WORK_DIR/worker.pid。 · start：2秒内返回0；worker持续运行并产生 heartbeat.log；重复启动不得产生第二个实例。 · status：运行时输出 running 并退出0，否则输出 stopped 并退出1。 · report：worker运行时退出0，生成 $WORK_DIR/resource.txt，包含 PID=<实际PID>、ps状态信息、top快照；未运行时退出1。 · stop：使 worker 正常终止并留下 stopped.log；没有运行时也成功。 · 未知命令：输出 usage 并退出 64。

2. 环境及工具清单

· 基础环境：WSL2 (Ubuntu) · 核心工具：/opt/recruitment/bin/slow-worker（后台程序） · Linux 命令：nohup、&、ps、top、kill、sleep、timeout、grep、export、for 循环、ps -p ... -o args=

3. 解题思路（核心流程与细节）

· 第一步：变量设置与函数封装。 使用兜底语法 export WORK_DIR="${WORK_DIR:-/workspace}" 设定工作目录，定义 PID_FILE 和 WORKER_BIN。编写 is_running() 函数，使用 kill -0 "$pid" 验证进程是否存在，再配合 ps -p "$pid" -o args= 和 grep 校验进程名，防止 PID 复用和僵尸进程。 · 第二步：start 单实例启动。 调用 is_running 检查单实例。启动时使用 nohup "WORKER_BIN" > /dev/null 2>&1 < /dev/null &，注意 < /dev/null 至关重要，防止后台进程因等待输入而挂起。启动成功后用 echo "!" > "$PID_FILE" 记录 PID。 · 第三步：status 状态查询。 完全基于 is_running 的返回码，输出 running 或 stopped。 · 第四步：report 资源快照。 使用 echo "PID=$pid" > "$report_file" 写入首行；追加 ps -p "$pid" -o pid,stat,args >> ...；追加 top -b -n 1 -p "$pid" >> ...（-b 批处理，-n 1 仅采样一次，-p 指定进程）。 · 第五步：stop 优雅终止与防超时。 发送 kill -TERM "$pid" 请求正常终止。关键点： 不能死循环等待，改用有限次 for 循环（如 5 次，每次 sleep 1，配合 timeout 保护 kill -0 探测），若超时仍未退出，再用 kill -9 兜底，最后清理 PID 文件并写入 stopped.log。

4. 我的疑问

· 疑问 1：什么叫“可重复使用的进程管理接口”？“实例”、“瞬时资源数值”、“真实状态”这些词到底什么意思？ · 疑问 2：export WORK_DIR="${WORK_DIR:-/workspace}" 这是什么语法？ · 疑问 3：ps -p "$pid" -o args= 和 grep -q "slow-worker" 这两个命令配合起来是干什么的？为什么不能只用 kill -0？ · 疑问 4：for 循环是怎么运行的？代码里哪里体现了“最多等五秒”？ · 疑问 5：[-n "$pid"] 是什么意思？

5. 遇到的问题及解决方法

【踩坑经历一】评测报错“脚本运行超时”

· 现象：脚本本地跑得通，一交评测就超时失败。 · 原因分析：一是后台程序启动时，终端可能会挂起等待输入；二是 stop 中使用了死循环等待进程退出，评测系统等不及直接判定超时。此外，start 的 sleep 时间过长也会导致超时。 · 解决方案：启动时加上 < /dev/null 彻底断绝输入依赖；将死循环改为有限次 for 循环；在探测命令前加上 timeout 0.5，强制命令在规定时间内返回，绝不无限等待。

【踩坑经历二】评测报错“worker 未在 TERM 后退出并留下记录”

· 现象：发送了 kill -TERM，但评测依然不通过。 · 原因分析：SIGTERM 是异步信号，进程收到后需要时间清理退出，之前的等待时间太短。如果进程最终被 kill -9 强制杀死，可能没有正确留下 stopped.log。 · 解决方案：在 stop 分支中，无论进程是正常退出还是被强制杀死，必须写入 echo "stopped" > "$WORK_DIR/stopped.log"，确保评测能检测到日志记录。

【踩坑经历三】无法判断进程是否真的活着

· 现象：PID 文件存在，但目标程序可能早就崩溃了，或者 PID 被系统分配给了其他无关进程。 · 原因分析：仅凭 kill -0 无法区分 PID 是否被复用，也无法判断是不是僵尸进程（状态为 Z）。 · 解决方案：在 is_running 函数中，先用 kill -0 判断，再用 ps -p "$pid" -o args= 获取进程真实命令行，用 grep 匹配程序名；同时检查状态是否为 Z（僵尸进程），排除虚假存活。 #第六关→交给服务账号----------

1. 题目详细描述

· 任务背景：需要编写 /workspace/setup-account.sh，建立符合要求的专用服务账号及文件访问状态，并依据 /workspace/task.txt 提取任务码生成 flag。 · 核心要求： · 系统组名为 archive，服务账号名为 archiver，账号 UID 范围为 200-999，主组为 archive，不创建 home 目录，登录 shell 为 /usr/sbin/nologin。 · 任务文件 /workspace/task.txt 的属主、属组分别为 archiver:archive。权限要求：所有者可读写，同组用户仅可读，其他用户无权限（即 640）。 · 预置的 visitor 用户必须不能读取该文件。archiver 必须能读取。 · 脚本必须可重复执行（幂等性）：已有符合要求的账号和组应复用；不符合要求的同名账号不能悄悄接受。 · 提取任务码并写入 /workspace/flag。

2. 环境及工具清单

· 基础环境：WSL2 (Ubuntu) · 核心工具：/workspace/task.txt（任务说明文件） · Linux 命令：getent、groupadd、useradd、id、chown、chmod、su、awk、cut、tr。

3. 解题思路（核心流程与细节）

· 第一步：创建组（幂等性处理）。 使用 getent group "$TARGET_GROUP" > /dev/null 2>&1 检查组是否存在。不存在才使用 groupadd 创建，避免重复执行时报错。 · 第二步：创建并校验服务账号。 使用 id "$TARGET_USER" 检查账号是否存在。若已存在，必须进行属性校验（UID 范围 200-999、主组为 archive、Shell 为 /usr/sbin/nologin）。任何一项不符，直接输出错误信息并 exit 1，绝对不将就。若不存在，使用 useradd -r -M -s /usr/sbin/nologin -g "$TARGET_GROUP" "$TARGET_USER" 创建。 · 第三步：精准设置文件权限。 使用 chown "$TARGET_USER:$TARGET_GROUP" "$TASK_FILE" 修改归属，使用 chmod 640 "$TASK_FILE" 设置权限（所有者读写、同组只读、其他无权限）。 · 第四步：以真实用户身份验证权限并提取任务码。 不依赖 ls -la 的视觉判断，而是使用 su -s /bin/bash -c "cat $TASK_FILE" "$TARGET_USER" 强制切换到 archiver 账号去实际读取文件。读取成功后，从 task.txt 中提取任务码写入 /workspace/flag。

4. 我的疑问

· 疑问 1：“shell设置为 /usr/sbin/nologin 是什么意思，shell不是一个脚本吗，还能设置吗？”（混淆了 Shell 脚本语言和登录 Shell 程序的概念）。 · 疑问 2：“if id "TARGET_USER" > /dev/null 2>&1; then ... USER_UID=(id -u ...) 这是什么意思？”为什么要查一堆属性？ · 疑问 3：“if ["$USER_UID" -lt 200] || ["$USER_UID" -gt 999]; then ... 这是什么意思？”为什么同名账号不能悄悄接受？ · 疑问 4：“cut: the delimiter must be a single character 这是什么意思？” · 疑问 5：“if su -s /bin/bash -c "cat $TASK_FILE" "$TARGET_USER" ... 这是什么意思？”为什么要费劲切换用户去读文件？

5. 遇到的问题及解决方法

【踩坑经历一】把 Shell 语言和登录 Shell 搞混

· 现象：不理解为什么要在 useradd 命令中加 -s /usr/sbin/nologin。 · 原因分析：误以为 Shell 只代表 bash 脚本。实际上，在 Linux 中 /usr/sbin/nologin 是一个被设计成“拒绝提供命令行交互”的登录程序。 · 解决方案：明白了服务账号需要极高的安全性。设置 nologin 后，即使密码泄露，攻击者也无法通过 SSH 登录系统。掌握用户管理命令：useradd 和 usermod 的 -s 参数。

【踩坑经历二】评测系统报错“不允许悄悄接受同名账号”

· 现象：如果系统里已经存在一个 UID 是 1001（普通用户）的用户名叫 archiver，脚本如果直接跳过创建，评测就会失败。 · 原因分析：题目要求服务账号 UID 必须在 200-999。同名但属性不符的账号很可能是恶意留下的，必须拒绝复用。 · 解决方案：在脚本中增加严格校验逻辑（检查 UID、主组、Shell）。一旦发现不符，立即 echo "Error..." 并 exit 1，坚决不将就。

【踩坑经历三】被一个中文全角冒号折腾崩溃

· 现象：脚本前面都跑通了，但提取任务码时总是报错：cut: the delimiter must be a single character。 · 原因分析：使用 od -c 检查文件发现，task.txt 里的冒号是中文全角冒号 ：。在 UTF-8 编码下，中文占 3 个字节，而 cut 命令只支持单字节分隔符。 · 解决方案：放弃使用 cut，改用 awk -F'：' '{print $2}' 替代，成功绕过字节限制提取出正确内容。同时，在非法参数处理中，用 rm -f "$ROUTE_RESULT" 确保“不生成结果文件”，完全匹配题目要求。

【踩坑经历四】只看 ls -la 就去提测

· 现象：明明用 chmod 640 改好了权限，但评测偶尔还是报错权限不对。 · 原因分析：提示3强调：“修改后用不同账号实际尝试访问，不能只依据显示的权限判断。” · 解决方案：在脚本中加入 su -s /bin/bash -c "cat ..." "$TARGET_USER" 这一步骤，切切实实让 archiver 用户去尝试读取，确保权限配置真实生效。如果失败则 exit 1，不将就。 #第七关→装好工具并读日志----------

1. 题目详细描述

· 任务背景：一个最小的 C 程序在当前环境中无法正常构建。需要修复构建环境、修改有瑕疵的源码，使其符合输出要求。 · 核心要求： · 编写 /workspace/setup-build.sh 记录可重复执行的环境配置（安装 build-essential 及其依赖）。 · 程序的源码在 /workspace/hello.c，构建入口为 /workspace/build.sh。 · 修复 hello.c：程序接收一个英文字母或数字组成的参数，正常退出时标准输出为 hello:<参数>，并向运行日志 run.log 追加 processed:<参数>，两者均含结尾换行。 · 基于修复后的代码，生成参数为 foundation 时的标准输出，写入 /workspace/flag。 · 通关条件：提交脚本、修复后的源码和 flag。评测会运行环境脚本两次（验证幂等性），确认 build-essential 已安装，并重新编译检查输出与日志。

2. 环境及工具清单

· 基础环境：WSL2 (Ubuntu) · 核心工具：apt-get（包管理）、gcc（C编译器）、build.sh（构建脚本） · Linux 命令：apt、cat、nano、重定向 > 和 >>、echo $?

3. 解题思路（核心流程与细节）

· 第一步：编写环境配置脚本。 编写 setup-build.sh，核心命令是 sudo apt update 和 sudo apt install -y build-essential。为了保证幂等性，使用 if ! command -v gcc 检查是否安装成功。加入 -y 参数防止安装过程卡死。 · 第二步：分析编译日志与修复源码。 运行 ./build.sh，它会将输出重定向到 build.log。查看日志发现 C 代码存在语法错误或逻辑缺失（例如未处理 argv 参数，或者缺少向文件追加的代码）。根据提示2，直接定位日志中的首个错误，使用 nano hello.c 修复代码（使用 printf 输出，并使用 fopen("/workspace/run.log", "a") 追加日志）。 · 第三步：重新编译并生成结果。 再次执行 ./build.sh，编译成功生成可执行文件。执行 ./hello foundation > /workspace/flag，将标准输出重定向为 flag 文件。

4. 我的疑问

· 疑问 1：build.sh 这个文件的作用是什么？里面的 set -euo pipefail 是什么意思？ · 疑问 2：为什么编译失败了，终端却没有任何输出？ · 疑问 3：为什么我要写一个 setup-build.sh 去安装 build-essential，它到底包含什么？ · 疑问 4：提示里说的“构建日志中首个编译错误往往比后续错误更有定位价值”是什么意思？

5. 遇到的问题及解决方法

【踩坑经历一】忽略了 -Werror 的严格性

· 现象：修复了 C 代码的语法后，依然编译失败，没有任何可执行文件。 · 原因分析：build.sh 中使用了 -Wall -Wextra -Werror。-Werror 会把所有的警告（比如定义了变量但没有使用、隐式类型转换）当作错误处理，直接拒绝生成文件。 · 解决方案：仔细检查 build.log，把代码中没有用到的变量删除，补全所有的 return 返回值，确保代码“一尘不染”，才通过了编译。

【踩坑经历二】不知道去哪里找报错信息

· 现象：执行 ./build.sh 后，屏幕干干净净，但就是没有生成程序。 · 原因分析：build.sh 的最后写了 > /workspace/build.log 2>&1，把标准输出和错误全存进文件了，屏幕上看不到。 · 解决方案：学会了编译失败后，第一时间执行 cat /workspace/build.log，掌握了阅读 C 语言编译日志的排错方法。

【踩坑经历三】日志追加与覆盖的区别

· 现象：程序运行后，run.log 里永远只有最后一次的记录。 · 原因分析：在 C 代码中使用 fopen 时，使用了 "w"（覆盖模式），而不是 "a"（追加模式）。 · 解决方案：将 C 代码中的文件打开模式改为 "a"，确保每次运行 ./hello 都能向日志末尾追加内容。 #第八关→git就在本地----------

1. 题目详细描述

.在 /workspace/repo 中执行 git init -b main，保持 notes.txt 原样且未暂存。也可从 /opt/recruitment/git-seed 克隆到 /workspace/repo。

2. 环境及工具清单

.基础环境：git bash .核心工具：cd，ls，cat，git init，git status，git diff

3. 解题思路

.先到对应目录用cat查看notes文件，初始化仓库，利用git status --short --branch和git diff --cached确认文件状态

#第九关→看清工作区和暂存区----------

1. 题目详细描述

.将 notes.txt 改为目标内容；只改工作区，不暂存、不提交,notes.txt 目标内容（最后一行带换行）： local git practice

first local note .通关条件：HEAD 保留 c0，索引保持基线，工作区符合目标。

2. 环境及工具清单

.基础环境：git bash .核心工具：cd，ls，cat，printf，git diff，git log，git status，>

3. 解题思路

.切到对应目录查看notes内容，利用printf将新内容覆写到notes里，利用git diff，git status，git log进行比较验证

4. 我的疑问

.疑问：为什么不可以用nano等编辑器？→解答：可能会自动加空格等

#第十关→留下第一个版本----------

1. 题目详细描述

.将 notes.txt 改为目标内容并提交，消息必须是 learn: first local commit。 .恰好一个以初始提交 c0 为父的新提交；HEAD 在 main，索引和工作区干净。

2. 环境及工具清单

.基础环境：git bash .核心工具：cd，ls，cat，printf，git add，git commit，git show，git diff，git status

3. 解题思路

.切到对应目录，cat查看文件，printf覆写修改内容，git diff确认修改，git add和git commit上传仓库

#第十一关→找回历史中的内容----------

1. 题目详细描述

.初始 c0 的 notes.txt 是 version one，c1 是 version two；当前工作区还有临时修改。恢复 c0 的内容，可用 restore 或 revert。restore 保留 HEAD=c1、索引=c1；revert 保留 c1 并新增一个恢复 version one 的提交，且干净。

2. 环境及工具清单

.基础环境：git bash .核心工具：git restore

3. 解题思路

.利用git diff，git status，git log查看不同，利用git restore恢复修改，最后用git diff，git status确认修改

#第十二关→让两条历史汇合----------

1. 题目详细描述

.从 c0 分出 main 和 topic。main 只将 main.txt 改为 main ready；topic 只将 topic.txt 改为 topic ready；各提交一次，再将 topic 合并到 main。各文件保留行末换行。 双亲合并保留分叉两侧内容；topic 指向其分支提交；main 的索引和工作区干净。

2. 环境及工具清单

.基础环境：git bash .核心工具：git branch，git switch，git merge

3. 解题思路

.用git branch创建分支topic，先对main下的notes进行修改，再switch切到topic分支，再对topic下的notes进行修改，最终利用merge进行合并，--no-edit是修改提示框不弹出，最后log，diff查看修改内容

#第十三关→完成可逆修改----------

1. 题目详细描述

.在 trial 分支将 notes.txt 改为目标内容并提交，消息 learn: first local commit；合入 main，然后用 revert 反向撤销。c0 → 修改 → 反向提交保留在 main；trial 指向修改提交；最终内容回到基线且干净。

2. 环境及工具清单

.基础环境：git bash .核心工具：git switch -c，git revert

3. 解题思路

.查看内容，用swich -c创建并切换到topic分支，printf覆写对topic下的notes进行修改，git add和commit提交，切到main分支，merge融合，revert新建未修改的情况，git log，git status查看修改内容

#进阶git→保留冲突两侧的意图----------

1. 题目详细描述

.main 和 topic 已经分叉修改 notes.txt。将 topic 合入 main，解决冲突，使文件依次为 keep main idea 与 keep topic idea 两行。双亲合并保留两侧修改，topic 保留原指针，main 的索引和工作区干净。

2. 环境及工具清单

.基础环境：git bash .核心工具：git add，printf

3. 解题思路

.查看两个分支下的notes内容，printf进行覆写，腹泻内容为两个notes的内容相加

#进阶HPC第一关→观察一个正在运行的程序----------

1. 题目详细描述

· 任务背景：在 Linux（或 WSL2）环境下，观察并监控一个正在运行的多线程程序。需要通过编译运行指定的 C++ 源码，启动目标进程，并利用采集器抓取其 CPU、线程和内存状态，最终生成标准化的数据报告。 · 核心要求： · 编译源码：手动下载 workload 与 collector 等源码及依赖，在 Linux 中编译生成可执行文件。 · 模拟负载与采集：先启动多线程 CPU 负载（workload），再采集目标进程的 CPU、线程和内存状态（collector）。 · 生成数据：提交采集器原样生成的 evidence/cpu-t8.jsonl 文件。 · 通关条件： · 提交 UTF-8 编码的 JSONL 文件。 · 文件内容必须包含：1 条 start、至少 15 条 sample、1 条 end。 · 时间覆盖至少 9 秒。 · 观察到多线程 CPU 负载。 · end.reason 字段必须为 duration_reached。

2. 环境及工具清单

· 基础环境：WSL2 (Ubuntu 系统) · 核心工具：CMake（项目构建工具）、Make（编译指挥工具）、G++（C++ 编译器） · Linux 命令：mkdir (建目录)、mv (移动文件)、cd (切换目录)、ls (查看文件)、head / tail (查看文件头尾)、wc -l (统计行数)。

3. 解题思路（核心流程与细节）

· 第一步：整理目录与构建编译（基于 CMakeLists.txt 与 src）。 · 按照工程规范，将下载的 .cpp 和 .hpp 源码统一放入 src 目录中，避免与构建产物混杂。 · 新建 build 目录作为构建车间。在 build 中执行 cmake .. 和 make，生成 workload 和 collector 两个可执行文件。 · 核心细节：CMakeLists.txt 中固定了源码路径（如 src/workload.cpp），这是构建成功的绝对前提。 · 第二步：启动多线程负载（基于 workload.cpp）。 · 打开终端 1，执行 ./workload。 · 核心细节：根据程序要求，必须传入 --duration 30s（指定运行时间及单位）、--threads 4（指定多线程数量），以及 --pid-file workload.pid（将当前进程的 PID 写入文件，供采集器读取）。 · 第三步：启动采集器采集数据（基于 collector.cpp 与 linux_procfs.cpp）。 · 打开终端 2，执行 ./collector。 · 核心细节：传入 --duration 20s（采集 20 秒）、通过 --pid-file 读取目标进程号，最后通过 --output ../evidence/cpu-t8.jsonl 指定输出路径。linux_procfs.cpp 在底层负责读取 /proc/[PID]/stat 等系统文件来获取具体指标。 · 第四步：数据校验与提交（基于 JSONL 格式要求）。 · 使用 head -n 5 和 tail -n 5 查看生成的 .jsonl 文件，确保有合法的 JSON 结构，且结尾的 reason 是 duration_reached。

4. 我的疑问

· 疑问 1：CMake 和 Make 到底发生了什么事？为什么非要建一个 build 文件夹？ · 解答：CMake 是看图纸（CMakeLists.txt）的“项目经理”，Make 是干活的“包工头”。建 build 文件夹是为了让构建过程产生的中间文件不弄脏纯源码，这叫“外部构建”。 · 疑问 2：为什么要分 .cpp 和 .hpp 文件？它们到底有什么用？ · 解答：.hpp 是“菜单”（声明），负责告诉别人有哪些功能；.cpp 是“后厨”（实现），负责具体的底层逻辑。分离它们是为了模块化和代码复用。 · 疑问 3：程序需要的参数（比如 --threads）只能靠报错试出来吗？ · 解答：不是。好的程序可以通过 --help 查看说明书，或者直接阅读 main 函数看源码是怎么解析参数的。当然，根据报错反推是最直接的手段。

5. 遇到的问题及解决方法

【踩坑经历一】未理解构建规则，导致找不到源码

· 现象：在终端执行 cmake，出现红色报错 Cannot find source file: src/workload.cpp。 · 原因分析：图纸（CMakeLists.txt）里写死要去 src 目录找材料，但我把源码全扔在了根目录，反而把构建目录命名成了 src。 · 解决方案：删除错误的 src 目录，新建 src 并把所有源码 mv 进去，再新建 build 进行编译。

【踩坑经历二】缺少时间单位后缀

· 现象：启动 workload 报错 duration must use an integer ms, s, or m suffix。 · 原因分析：直接输入 --duration 30，程序不知道是 30 毫秒、30 秒还是 30 分钟。 · 解决方案：加上单位后缀，改为 --duration 30s。

【踩坑经历三】未满足多线程的条件，导致启动失败

· 现象：改好时间后，报错 --threads must be in 1..256。 · 原因分析：题目要求"多线程负载"，但启动命令漏了指定线程数的参数。 · 解决方案：补齐参数，加上 --threads 4。

【踩坑经历四】时序错乱，PID 文件神秘消失

· 现象：采集器运行报错 cannot open PID file: No such file or directory。 · 原因分析：终端 1 的 workload 只运行了 30 秒，我手速太慢，切到终端 2 时 workload 已经执行完毕并自动删除了 workload.pid 文件。 · 解决方案：将 workload 的运行时间延长至 60s，确保在它存活期间，迅速去另一个终端启动 collector 读取 PID 文件。 #进阶HPC第二关→判断性能瓶颈并提出优化方案----------

1. 题目详细描述

· 任务背景：这是一个旨在判断程序性能瓶颈并提出优化方案的高性能计算（HPC）实验。要求在 Linux 环境下，针对 Stencil（模板计算）和 GEMM（矩阵乘法）两个核心算法，对比不同访存顺序和分块策略对性能的影响。 · 核心要求： · 下载并手动编译 stencil.cpp、gemm.cpp 及相关依赖文件，编译参数必须保持一致（使用 -O3 -march=native -std=c++17）。 · 对比测试 Stencil 的按行（Row）与按列（Column）访问，以及 GEMM 的朴素实现（Naive）与分块实现（Blocked）。 · 推导并给出计算量、内存流量模型，计算算术强度。 · 撰写实验报告，必须包含：正确性验证、平均时间、中位数、总体标准差、GFLOPS/s 和有效带宽（Stencil按理想空间复用计算）。 · 基于数据和 Roofline 模型判断性能瓶颈，并给出优化建议。 · 通关条件：不设在线评测。需在 Linux 完成实验并提交包含公式推导、对比数据、瓶颈分析和优化决策的完整实验报告（最终以 PDF 格式提交）。

2. 环境及工具清单

· 基础环境：Linux 虚拟机（终端环境）。 · 核心工具： · g++：C++ 编译器（用于将代码翻译为机器指令）。 · common.hpp：提供的公共工具包（内含计时器、数据初始化、结果验证、指标计算等功能）。 · stencil.cpp 与 gemm.cpp：实验的核心源码文件。 · 编译指令参数：-O3（最高级别优化）、-march=native（调用本机最高级 SIMD 指令集如 AVX2/AVX512）、-std=c++17（使用 C++17 标准）。

3. 解题思路（核心流程与细节）

· 第一步：环境配置与代码编译 使用终端进入代码所在目录（如 hpc_lab）。输入 g++ -O3 -march=native -std=c++17 stencil.cpp -o stencil 和对应的 gemm 编译命令。这一步核心是将人类可读的 .cpp 翻译成机器可执行文件，不包含 .hpp 文件（由主程序自动引用）。 · 第二步：理解程序参数并运行实验 直接执行 ./stencil 或 ./gemm 会输出用法提示（usage）。根据提示，执行如 ./stencil 1000 1000 100 row 10（高度、宽度、步数、遍历方式、重复次数）和 ./gemm 1024 blocked 32 10（矩阵大小、算法模式、分块大小、重复次数）。程序会自动进行 warmup=1（预热）并记录多轮数据（平均、中位、标准差、GUpdates/s 或 GFLOPS）。 · 第三步：理论推导与数据对比分析 利用公式推导理论 FLOPs 和内存流量（Bytes），计算算术强度（AI = FLOPs / Bytes）。对比 Row 与 Column 发现后者慢数倍（空间局部性极差）；对比 Naive 与 Blocked 发现后者快 30 倍（分块提升了时间局部性，减少了 Cache Miss）。 · 第四步：撰写并导出实验报告 将理论公式、实测数据表、瓶颈分析（结合 Roofline 模型判断是带宽受限还是计算受限）、优化建议撰写成 Markdown 或 Word 文档，最终导出为 PDF 提交。

4. 我的疑问

· 疑问 1：common.hpp 文件是干什么用的？为什么不用单独编译它？ · 解答：它是“公共工具包”（头文件）。里面封装了计时、结果校验等通用函数，供 .cpp 主文件调用，相当于公共菜谱，不需单独执行。 · 疑问 2：编译参数里的 -O3 和 -march=native 到底是什么意思？为什么必须加？ · 解答：-O3 是让编译器拼命优化代码速度；-march=native 是让编译器根据当前电脑的硬件配置，自动使用最快、最高级的指令集（类似给大厨配上高级多孔切菜器），这对矩阵计算性能影响巨大。 · 疑问 3：为什么屏幕输出里有一大串 check=PASS max_abs_error=... checksum=...？ · 解答：这是 common.hpp 里的验证工具。它在检查优化后的代码算出来的结果对不对，PASS 代表结果正确，没有因为盲目提速而算错。 · 疑问 4：提示里说“模型预测不等于真实硬件测量”，是什么意思？ · 解答：公式算出来的理论最快速度（Roofline 天花板）在现实中是达不到的，因为实际运行中有内存延迟、缓存未命中（Cache Miss）、系统卡顿等损耗，这也是我们做性能测试和优化的意义所在。

5. 遇到的问题及解决方法

【踩坑经历一】输入参数超出程序限制导致报错

· 现象：在运行 GEMM 的 Naive 模式时，输入了命令 ./gemm 1024 naive 0 10，程序直接报错罢工。 · 原因分析：程序内设置了参数边界检查。虽然说明里写了“Naive 会忽略 tile 大小”，但检查机制仍要求 tile 的值必须在 1~256 之间，填 0 触发了安全拦截。 · 解决方案：将参数改为合法值，如 ./gemm 1024 naive 32 10，成功运行。

【踩坑经历二】不知道怎么运行程序

· 现象：第一次敲完 ./stencil 后，屏幕只输出了几行英文（usage/limits），程序并没有开始计算。 · 原因分析：程序需要接收参数才能运行（Height, Width 等），没带参数时，它只是在提示你怎么使用。 · 解决方案：仔细阅读了输出的 usage 提示，按格式输入了 ./stencil 1000 1000 100 row 10，程序立刻开始正常跑数据。 #进阶HPC第三关→日志压缩与稳定筛选优化----------

1. 题目详细描述

· 任务背景：在高性能计算（HPC）场景下，原始的单线程日志筛选程序（baseline.cpp）处理大规模数据时存在性能瓶颈。需要在 Linux 环境下使用 C++17 实现稳定筛选（保持输入顺序），并进行性能优化，最后提交实验报告和符合规范的源码包。 · 核心要求： · 接口抽离：必须从 baseline.cpp 中提取出 std::size_t filter_records(const Record* input, std::size_t count, Record* output, const Query& query) 接口，该函数内绝对不能包含 I/O、打印、内存分配或生成数据操作。 · 正确性与稳定性：筛选出的日志数量、字段及顺序必须与原始输入完全一致（即使是多线程处理，也不能乱序）。 · 性能优化与对比：至少测试两种不同的数据规模，预热后测量至少 5 次，保留原始样本，并在报告中分析优化的适用条件及退化场景。 · 打包规范：提交单根目录的 .tar.gz 压缩包，不可包含绝对路径、软链接、构建产物（如 .o、可执行文件）或隐藏测试。

2. 环境及工具清单

· 基础环境：Ubuntu 64位（VMware 虚拟机，强制分配 8 线程测试）。 · 核心工具：g++（C++17编译器）、python3（用于运行公开测试脚本）、tar（打包工具）。 · Linux 命令与参数：g++ -O3 -std=c++17 -pthread（编译命令）、python3 test_public.py ./executable（带参运行测试）、ls -lh（查看文件大小）、tar -czvf（打包）。 · 代码工具：C++ 标准库中的 （多线程）、（计时）、（算法处理）。

3. 解题思路（核心流程与细节）

· 第一步：环境构建与基线跑通 · 细节：运行 g++ -O3 -std=c++17 baseline.cpp -o baseline_test 编译原始代码。发现运行 test_public.py 需要传入可执行文件路径，使用 python3 test_public.py ./baseline_test 跑通 7 个

测试用例，确认基线环境无误。 · 第二步：抽离核心接口（正确性保障） · 细节：将原本位于 main 函数 BEGIN CORE 和 END CORE 之间的循环剥离，放入独立的 filter_records 函数中。核心代码如下：

```
for (std::size_t i = 0; i < count; ++i) {
    const Record& r = input[i];
    if (query.begin <= r.timestamp && r.timestamp < query.end &&
        r.latency_us >= query.min_latency_us &&
        (r.flags & query.required_flags) == query.required_flags) {
        output[matched++] = r; // 顺序遍历写入，保证稳定
    }
}
```

· 在 main 中替换为调用该函数，重新编译并跑通测试，拿到及格分。 · 第三步：多线程分块并行优化（性能提升） · 细节：引入 std::thread。核心逻辑分为三个阶段： 1. 并行统计：划分 8 个数据块，各线程只统计自己块内命中的数量，存入 counts 数组。 2. 计算前缀和：计算 offsets 数组，确定每个线程在 output 中应该从第几个位置开始写。 3. 并行写入：各线程带着自己的 offset 再次遍历数据块，将命中的记录直接写入 output 的指定位置。这保证了既快又绝对不乱序（无写冲突）。 · 第四步：性能测试与规范打包 · 细节：用脚本跑 10 万条和 100 万条数据进行 5 次测量取中位数。将结果填入 report.md。新建干净目录，只保留 baseline.cpp、support.hpp、test_public.py、报告、CMakeLists.txt 等，使用 tar -czvf submission.tar.gz 打包。

4. 我的疑问

· 疑问 1：题目资料里根本没有 README.md 文件，参数和标准去哪里找？ · 解答：不用慌，直接从 baseline.cpp 的默认参数、support.hpp 的结构体定义以及 test_public.py 的测试用例中反推。在实验报告中如实说明，反而能体现逆向工程能力。 · 疑问 2：编译成功了，但运行 python3 test_public.py 没有反应，只提示提供路径？ · 解答：测试脚本本身不知道你的程序叫什么名字，必须把编译好的可执行文件作为参数传给它，例如 ./baseline_test。 · 疑问 3：多线程代码明明写了，为什么终端打印出来的日志永远是 threads=1？ · 解答：这是 baseline.cpp 原始代码里 main 函数 std::cout 写死的一串字符串，不是真正的线程数。为了真实性，后来在代码中手动修改了打印内容。 · 疑问 4：虚拟机明明分配了 16 个核，为什么多线程不仅没变快，还变慢了？ · 解答：虚拟机的 hardware_concurrency() 经常识别不准，可能退化成了单线程执行，但依然承担了创建线程和分配内存的开销。后来强制写死 num_threads=8 才真正测出效果。

5. 遇到的问题及解决方法

【踩坑经历一】多线程引起的“负优化”现象

· 现象：在使用 8 线程优化后，跑 10 万条数据（2.55ms）竟然比单线程基线（0.40ms）慢了足足 6 倍；但在跑 100 万条数据时（3.49ms），又比单线程（4.34ms）快。 · 原因分析：线程的创建、销毁、vector 数组分配是有固定开销的。数据量小的时候（10万条计算极快），固定开销压倒了并行计算的收益。 · 解决方案：在实验报告的“性能与适用条件”部分，如实记录了这个拐点现象。指出该多线程方案仅适用于百万级以上的大规模数据，在小数据规模下应退化为单线程。

感悟报告

1. linux真是一切皆文件，之前预习的时候还没觉得有什么，实际上手尝试做题后发现，无论是linux的环境还是日志的表现形式都是以文件的格式存在的，就连报错日志都需要追加到日志里，而不是单纯打印到屏幕上，无论是这次考核要求的通关形式还是网上的相关视频，Linux环境中总是严格按文件形式分好类，格式干净整洁有序

2. 之前预习时背下来了linux的相关命令，但实际上手发现远没有那么简单，无论是查看还是操作，都会有更合适的命令在里面，如各种减号后缀代表的各种意思
3. ai的使用是这次学习最大的占比了，也让我深刻意识到ai使用并不是一项无用的技术，反而是保证对话高效有序进行的一门学问，诸如让语言变得通俗易懂，根据自身水平建立讲解模式，还是代码讲解的表现形式，用好ai都能节省大量的时间精力，就比如因为我给ai的截图不清晰导致中文与英文搞混排查错误又浪费了大把的时间，还是说提示词表述不清导致ai答复答非所问
4. 相关语言术语也是在这次考核中给予我颇深理解障碍的环节，就比如linux相关题目中要求我严格以二进制比对，尝试用nano直接进行修改反而无法通关，原因是nano自带的换行符，又比如shell脚本编译中那些晦涩难懂的专业术语，工作目录，接口，结果文件路径，配置文件，进程号文件，实例，瞬时资源数值，真实状态等，对我理解题目造成了很大的阻碍，让我再次意识到基础的关键不仅在于用于做题，还是用于题目理解
5. 进程之间是分离的，也是让我印象深刻的一点，无论是第四关出现的export将变量看给子进程还是hoc中第一关c语言收集信息是双开进程，都表现了进程的分离性
6. 严格的比较性，linux中有一题要求运行test，我直接使用workspace下的test.bin却无法通关，证实了文件名称的严格比较
7. 中文字节与英文字节的非一致性，在shell中一题中，我尝试grep字符然后cut切割选取我想要的字符，但是却失败了，原因就是我用中文冒号作为分隔符，占三字节，而cut只允许一字节
8. 性能优化的检验，之前我一直以为hpc是只有python，这次考核告诉我性能优化绝不止一种编程语言，在这次性能优化效能的检验中，我意识到检验性能的绝不止一种指标，除去python中的加速比和并行效率，还有平均时间，中位数，总体标准差，计算吞吐，gflops，有效带宽等，threads的不同也会有不同的实验效果，可能存在反直觉的性能结果
9. c和python的算法优化相似性，刚接触c语言的性能优化，我是晕头转向，但在比较有意深入了解后，发现与python也是有许多相似性的地方，比如多进程，矩阵乘法等，这帮助我更快的上手c语言的性能优化，即便是在c基础还在打情况下
10. 无论是shell，linux还是hpc性能优化，都考眼了对逻辑思维的严格考察，从在与ai问答反复提及“这串代码想表达的意思，在整串代码中的作用”就能看出来逻辑的严格重要性，比如一道shell脚本的题要求在src文件夹里cmake，我放在了build里，理所当然的报错了，cd的位置，cmake的对象状态都给予了我很大启发
11. 信息的收集，无论是读懂题目，看懂脚本，信息的收集都至关重要，再次拿第十点的例子打比方，错误的主要原因其实还是信息收集的不完全，这启示我在做题前，要京可能的收集给的一切信息，无论是cat查看cmakelist文件还是--help查看函数或程序的使用规则和传入参数，都会比报错后查看报错信息要来的高速快捷，这也同样启示使用ai要给他喂足够量的信息，就比如hpc第三题，没有足够的信息，ai是无法解出性能优化方案的
12. git的团队协作，从前我一直觉得团体写代码一直很高大上并且遥不可及，但实际体验后发现，其实每次修改都有迹可循，修改冲突也是最朴实无华的重新覆写，让我减轻了对git协作方面的困惑，有协作的动力

这次考核给了我一个沉浸于逻辑语言艺术美感的机会，我花费了大量时间从一个小白成长为了大白（还是很菜）（），但也是实实在在感受到了学习这一方面的乐趣，看着这一段时间的笔记也让我颇有成就感，让我意识到兴趣也是进步的一大动力，